

CHƯƠNG 2 HỌC PYTHON CHO TRÍ TUỆ NHÂN TẠO

Như chúng ta đã thấy Trí tuệ Nhân tạo có thể hữu ích như thế nào trong nông nghiệp nhưng câu hỏi tiếp theo luôn xuất hiện trong đầu là chúng ta có thể bắt đầu học từ đâu, bằng cách nào và các mô hình học một trí tuệ nhân tạo là gì.

Vì Trí tuệ Nhân tạo là một phần rất lớn của Khoa học Máy tính, điều này có nghĩa là chúng ta cần một ngôn ngữ làm phương tiện để thực hiện nó và chúng ta luôn ưu tiên Python cho việc đó, một ngôn ngữ rất linh động và dễ học.

Trong chương này, chúng ta sẽ học về những kiến thức cơ bản của python để nó sẽ hữu ích cho chúng ta trong việc triển khai trí tuệ nhân tạo sau này.

I. Giới thiệu

Python là một ngôn ngữ đáng chú ý, vừa dễ học vừa mạnh mẽ. Bạn sẽ ngạc nhiên khi thấy thật đơn giản để không phải tập trung vào ngôn ngữ bạn đang lập trình, mà là tập trung vào việc giải quyết vấn đề.

Guido van Rossum là người tạo ra ngôn ngữ Python, được đặt tên theo chương trình “Monty Python’s Flying Circus” của đài BBC.

Python được giới thiệu chính thức là:

“Python là một ngôn ngữ lập trình mạnh mẽ, dễ học. Nó có các cấu trúc dữ liệu cấp cao hiệu quả và một cách tiếp cận đơn giản nhưng hiệu quả với lập trình hướng đối tượng. Cú pháp tao nhã và kiểu dữ liệu động của Python, cùng với bản chất thông dịch, làm cho nó trở thành ngôn ngữ lý tưởng cho việc viết kịch bản và phát triển ứng dụng nhanh trong nhiều lĩnh vực trên hầu hết các nền tảng.”

II. Các đặc điểm của Python

- **Đơn giản:** Ngôn ngữ Python đơn giản và tinh gọn. Đọc một chương trình Python tốt gần giống như đọc tiếng Anh, nhưng thực sự rất cô đọng! Việc Python giống như mã giả là một trong những tài sản lớn nhất của nó. Điều này giúp bạn tập trung vào giải pháp cho vấn đề thay vì bản thân ngôn ngữ.
- **Dễ học:** Như bạn sẽ thấy, Python có thể được bắt đầu rất nhanh. Như đã mô tả, Python có cú pháp cực kỳ đơn giản. * **Miễn phí và mã nguồn mở:** Python là một ví dụ về FLOSS (Phần mềm Tự do và Mã nguồn mở). Bạn có thể chia sẻ các bản sao miễn phí, đọc mã nguồn của phần mềm, thực hiện các thay đổi quan trọng và sử dụng một phần của nó trong các chương trình miễn phí mới. FLOSS dựa trên ý tưởng về một mạng lưới chia sẻ trí tuệ. Đây là một trong những yếu tố Tại sao Python được phát triển và liên tục thay đổi.

- **Ngôn ngữ cấp cao:** Bạn không bao giờ phải bận tâm đến các thông tin nhỏ nhất như quản lý việc sử dụng bộ nhớ trong chương trình của mình, v.v., khi bạn lập trình bằng Python.
- **Khả chuyển (Portable):** Python đã được điều chỉnh cho (tức là, cập nhật cho) nhiều nền tảng vì nó là mã nguồn mở. Bất kỳ hệ thống nào trong số này cũng sẽ chạy tất cả các chương trình Python của bạn mà không cần sửa đổi nếu bạn đủ cẩn thận để tránh các chức năng phụ thuộc vào hệ thống. Python cũng có thể được sử dụng cho GNU/Linux, Windows, FreeBSD, Macintosh, Solaris, OS/2, Amiga, AROS, AS/400, BeOS, OS/390, z/OS, Palm OS, Windows CE và PocketPC. Bạn cũng có thể tạo trò chơi cho máy tính, iPhone, iPad hoặc Android của mình với một nền tảng như Kivy.
- **Khả năng mở rộng (Extensible):** Bạn có thể lập trình một phần của chương trình bằng C hoặc C++ và sau đó sử dụng nó từ chương trình Python của mình nếu bạn cần một phần mã quan trọng chạy thật nhanh hoặc bạn muốn một số thuật toán không bị tiết lộ.
- **Khả nhúng (Embeddable):** Để cung cấp cho người dùng khả năng viết kịch bản cho phần mềm của bạn, bạn có thể tích hợp Python vào các chương trình C/C++ của mình.
- **Thông dịch (Interpreted):** Một chương trình được biên dịch như C hoặc C++ được dịch từ ngôn ngữ nguồn, tức là C hoặc C++ sang ngôn ngữ lập trình (mã nhị phân tức là 0 và 1) bằng một trình biên dịch chứa các cờ và tùy chọn khác nhau. Phần mềm Linker/Loader (Trình liên kết/Trình tải) sao chép và bắt đầu chạy chương trình này từ đĩa cứng vào bộ nhớ. Mặt khác, Python không yêu cầu biên dịch sang mã nhị phân. Phần mềm được chạy trực tiếp từ mã nguồn. Python tự động chuyển đổi mã nguồn thành một loại trung gian gọi là mã byte (bytecodes), sau đó dịch nó sang ngôn ngữ gốc của máy tính rồi thực thi. Trên thực tế, sử dụng Python dễ dàng hơn nhiều vì bạn không phải lo lắng về việc biên dịch phần mềm để đảm bảo các thư viện chính xác được đính kèm và tải, v.v. Điều này làm cho chương trình Python của bạn dễ chuyển đổi hơn, vì bạn chỉ cần sao chép Python của mình sang một máy khác và nó cứ thế hoạt động!
- **Hướng đối tượng (Object Oriented):** Python hỗ trợ lập trình hướng thủ tục cũng như lập trình hướng đối tượng. Trong các ngôn ngữ hướng thủ tục, phần mềm được xây dựng xung quanh các thủ tục hoặc hàm, vốn chỉ là những đoạn chương trình chung. Trong các ngôn ngữ hướng đối tượng, phần mềm được thiết kế xung quanh các đối tượng kết hợp dữ liệu và chức năng. So với các ngôn ngữ chính như C++ hoặc Java, Python có một cách thức tốt nhưng đơn giản hóa để thực hiện OOP (Lập trình hướng đối tượng).
- **Thư viện phong phú:** Thư viện cơ bản của Python vô cùng lớn. Nó sẽ giúp bạn thực hiện nhiều việc khác nhau với biểu thức chính quy (regular expressions), tạo tài liệu, phân tích đơn vị (unit analysis), phân luồng (threading), cơ sở dữ liệu, trình duyệt web, CGI, FTP, email, XML, XML-RPC, HTML, WAV, mã hóa, GUI (Giao diện đồ họa người dùng) và các đối tượng phụ thuộc hệ thống khác. Tất cả

những điều này đều được tích hợp sẵn ở bất cứ đâu Python được cài đặt. Điều này được biết đến với triết lý “pin kèm theo” (batteries included) của Python. Bên cạnh thư viện chính, Chỉ mục Gói Python (Python Package Index) cũng cung cấp nhiều thư viện chất lượng cao khác.

III. Cài đặt

Python có bản phân phối cho một số lượng lớn các nền tảng. Bạn chỉ cần tải xuống mã nhị phân (binary code) cho ứng dụng của mình và cài đặt Python.

Nếu mã nhị phân cho nền tảng của bạn không có sẵn, bạn cần một trình biên dịch C để biên dịch mã nguồn bằng tay. Bạn có thể biên dịch mã nguồn một cách linh hoạt hơn để đưa ra lựa chọn về các tính năng cần thiết trong quá trình cài đặt.

Dưới đây là mô tả ngắn gọn về việc cài đặt Python trên các nền tảng khác nhau -

- **Cài đặt trên Unix và Linux**

- Hãy làm theo các bước sau để cài đặt Python trên máy Unix/Linux.
- Mở trình duyệt Web và truy cập <https://www.python.org/downloads/>.
- Làm theo liên kết để tải xuống mã nguồn đã nén có sẵn cho Unix/Linux.
- Tải xuống và giải nén các tệp.
- Chỉnh sửa tệp Modules/Setup nếu bạn muốn tùy chỉnh một số tùy chọn.
- chạy tập lệnh ./configure
- make
- make install
- Thao tác này sẽ cài đặt Python tại vị trí tiêu chuẩn /usr/local/bin và các thư viện của nó tại /usr/local/lib/pythonXX, trong đó XX là phiên bản của Python.

- **Cài đặt trên Windows**

- Truy cập <https://www.python.org/downloads/> và tải xuống phiên bản mới nhất. Tại thời điểm viết bài này, đó là Python 3.7 hoặc phiên bản cao hơn. Việc cài đặt cũng giống như bất kỳ phần mềm nào khác chạy trên Windows.
- Lưu ý rằng nếu phiên bản Windows của bạn là pre-Vista (trước Vista), bạn chỉ nên tải xuống Python 3.4 vì các phiên bản mới hơn yêu cầu các phiên bản Windows mới hơn.
- **THẬN TRỌNG:** Hãy chắc chắn rằng bạn đã chọn tùy chọn **Add Python 3.7 or greater version to PATH** (Thêm Python 3.7 hoặc phiên bản cao hơn vào PATH).
- Để thay đổi vị trí cài đặt, hãy nhấp vào **Customize installation** (Tùy chỉnh cài đặt), sau đó nhấp vào **Next** và nhập C:\python35 (hoặc một vị trí thích hợp khác) làm vị trí cài đặt.

- Nếu trước đó bạn không chọn tùy chọn **Add Python 3.7 or greater version PATH** (Thêm Python 3.7 hoặc phiên bản cao hơn vào PATH), hãy chọn **Add Python to environment variables** (Thêm Python vào biến môi trường). Tùy chọn này thực hiện điều tương tự như **Add Python 3.7 or greater version to PATH** trên màn hình cài đặt đầu tiên.
- Bạn có thể chọn cài đặt **Launcher** (Trình khởi chạy) cho tất cả người dùng hoặc không, điều đó không quan trọng lắm. Launcher được sử dụng để chuyển đổi giữa các phiên bản Python khác nhau đã được cài đặt.
- Nếu PATH của bạn không được thiết lập đúng (bằng cách chọn tùy chọn **Add Python 3.7 or greater version Path** hoặc **Add Python to environment variables**), thì hãy làm theo các bước trong phần tiếp theo (DOS Prompt) để khắc phục. Nếu không, hãy chuyển đến phần **Running Python prompt on Windows** (Chạy dấu nhắc Python trên Windows) trong tài liệu này.
- Lưu ý: Dành cho những người đã biết lập trình, nếu bạn đã quen thuộc với Docker, hãy xem qua Python trong Docker và Docker trên Windows.
- **Cài đặt trên Mac OS**
 - Nếu bạn đang dùng Mac OS X, bạn nên sử dụng **Homebrew** để cài đặt Python 3. Đây là một trình cài đặt gói tuyệt vời cho Mac OS X và nó thực sự dễ sử dụng.
 - Nếu bạn chưa có Homebrew, bạn có thể cài đặt nó bằng lệnh sau:
`ruby -e "(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/master/install)"`
 - Chúng ta có thể cập nhật trình quản lý gói bằng lệnh bên dưới:
`brew update`
 - Bây giờ hãy chạy lệnh sau để cài đặt Python3 trên hệ thống của bạn:
`brew install python3`

IV. Thiết lập PATH

Các chương trình và các tệp thực thi khác phải nằm trong nhiều thư mục khác nhau để cung cấp cho hệ điều hành một đường dẫn tìm kiếm liệt kê các thư mục mà HĐH tìm kiếm.

Đường dẫn (route) được lưu trữ dưới dạng một chuỗi có tên được duy trì bởi hệ điều hành trong một biến môi trường. Biến này được cung cấp cho Shell lệnh và các chương trình khác.

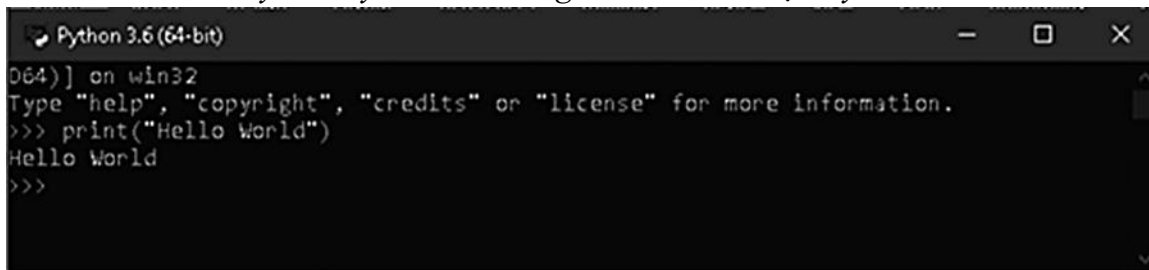
Đối với Unix và Windows, thuộc tính đường dẫn được gọi là PATH (Unix phân biệt chữ hoa chữ thường; Windows thì không). Trình cài đặt sẽ quản lý thông tin theo dõi trong Mac OS. Bạn phải thêm thư mục Python vào đường dẫn của mình để gọi trình thông dịch Python từ bất kỳ thư mục cụ thể nào.

- **Thiết lập Path tại Unix/Linux**

- Để thêm thư mục Python vào đường dẫn cho một phiên cụ thể trong Unix
- **Trong csh shell** - gõ setenv PATH "PATH:/usr/local/bin/python" và nhấn Enter.
- **Trong bash shell (Linux)** - gõ export PATH="PATH:/usr/local/bin/python" và nhấn Enter.
- **Trong sh hoặc ksh shell** - gõ PATH="PATH:/usr/local/bin/python" và nhấn Enter.
- *Lưu ý: /usr/local/bin/python là đường dẫn của thư mục Python.*

- **Thiết lập Path tại Windows**

- Để thêm thư mục Python vào đường dẫn cho một phiên cụ thể trong Windows -
- Tại dấu nhắc lệnh (command prompt) - gõ path %path%;C:\Python và nhấn Enter.
- *Lưu ý: C:\Python là đường dẫn của thư mục Python.*



```
Python 3.6 (64-bit)
064)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello World")
Hello World
>>>
```

Fig. 2.1 Python Example in IDE

V. Chạy Python với GUI

Nếu bạn có GUI (Giao diện đồ họa người dùng) trên máy của mình hỗ trợ Python, bạn cũng có thể chạy Python từ môi trường GUI.

- **Unix:** IDLE là IDE (Môi trường phát triển tích hợp) đầu tiên trên Unix dành cho Python.
- **Windows:** PythonWin là giao diện Windows đầu tiên cho Python và là một IDE có GUI.
- **Macintosh:** Phiên bản Python dành cho Macintosh cùng với IDLE IDE có sẵn trên trang web chính, có thể tải xuống dưới dạng tệp MacBinary hoặc BinHex'd.

Bạn nên nhờ sự hỗ trợ từ người quản trị hệ thống nếu không thể thiết lập framework một cách chính xác. Đảm bảo rằng hệ thống Python được thiết kế đúng cách và chạy tốt.

Một trang web khác tên là Anaconda có thể được sử dụng. Chúng cũng là mã nguồn mở. Nó bao gồm hàng trăm gói khoa học dữ liệu phổ biến, gói conda và trình quản lý môi trường ảo cho Windows, Linux và MacOS. Bạn có thể truy cập nó từ liên kết <https://www.anaconda.com/download/> bằng hệ điều hành của mình.

Trong cuốn sách này, tất cả các chương trình đều sử dụng Python 3.6 trên MS Windows 10.

VI. Bắt đầu với python

(a) Bước đầu tiên:

Bây giờ chúng ta sẽ xem cách chạy một chương trình 'Hello World' tiêu chuẩn trong Python. Bạn sẽ học cách viết, lưu và chạy các chương trình Python. Có hai phương pháp sử dụng Python để chạy phần mềm: dùng dấu nhắc thông dịch tương tác (interactive interpreter prompt) hoặc sử dụng tệp nguồn (source file). Bây giờ chúng ta hãy xem cách sử dụng hai cách tiếp cận này.

Bằng Dấu nhắc Thông dịch (Interpreter Prompt):

Mở terminal (cửa sổ dòng lệnh) trong hệ điều hành của bạn và sau đó mở dấu nhắc Python bằng phím python3. Phím [enter] sẽ được nhấn và mở ra.

Bạn sẽ thấy dấu >>> khi bạn bắt đầu gõ nội dung sau khi đã khởi động Python. Đây được gọi là *dấu nhắc trình thông dịch python* (python interpreter prompt).

Tại dấu nhắc trình thông dịch Python, gõ:

```
>>print ("Hello World")
```

Sau đó nhấn phím [Enter] và nó sẽ đưa ra kết quả -> Hello World.

Đây là một ví dụ về giao diện màn hình sẽ trông như thế nào trong Hình 2.1

Thoát khỏi Dấu nhắc Thông dịch

Bạn có thể thoát khỏi trình thông dịch bằng cách nhấn [ctrl + d] hoặc nhập exit() (nếu bạn đang sử dụng shell GNU/Linux hoặc OS X (lưu ý: nhớ sử dụng dấu ngoặc đơn, ())), sau đó nhấn phím [enter].

Trên dấu nhắc lệnh/trình thông dịch của Windows, nhấn [ctrl+z] rồi nhấn [enter].

Sử dụng Trình chỉnh sửa (Editor)

Trình chỉnh sửa mã (code editor) là một công cụ để viết và chỉnh sửa mã. Chúng thường dễ làm quen và có thể rất tuyệt. Tuy nhiên, khi phần mềm của bạn lớn hơn, bạn sẽ phải kiểm tra và gỡ lỗi (debug) mã của mình.

IDE (môi trường phát triển tích hợp) giúp hiểu mã của bạn tốt hơn nhiều so với một trình soạn thảo văn bản (text editor). Nó thường bao gồm các tính năng như tự động hóa cấu trúc (construct automation), bố cục ứng dụng (application layout), kiểm thử (testing) và gỡ lỗi (debugging). Nó sẽ cải thiện đáng kể việc nghiên cứu của bạn. Nhược điểm là các IDE rất khó sử dụng.

Có rất nhiều Trình chỉnh sửa như python IDE, Atom, PyCharm, Visual Studio code, v.v.. Chúng có thể được sử dụng cho mục đích lập trình và phát triển các dự án khác nhau.

(b) Những điều cơ bản:

Chỉ in ra “hello world” là không đủ, phải không? Bạn muốn làm điều gì đó khác - như thực hiện một số thay đổi, sử dụng chúng và nhận lại kết quả. Trong Python, điều này có thể được thực hiện bằng cách sử dụng hằng số và biến, và trong chương này, một số khái niệm khác cũng sẽ được thảo luận.

BÌNH LUẬN (COMMENTS)

Bình luận là văn bản nằm ở bên phải ký hiệu # và thường hữu ích cho người đọc về mặt kỹ thuật. Ví dụ:

```
>>>print ('hello world') # Lưu ý rằng print là một hàm
```

hoặc:

```
# Lưu ý rằng print là một hàm  
>>>print ('hello world')
```

Hãy sử dụng càng nhiều bình luận hữu ích càng tốt để làm rõ các quyết định quan trọng trong chương trình của bạn, làm rõ các dữ kiện quan trọng, giải thích các vấn đề bạn đang cố gắng giải quyết, tương ứng.

Code cho bạn thấy **làm thế nào** (how), bình luận giải thích **tại sao** (why).

HẰNG SỐ CỤ THỂ (LITERAL CONSTANTS)

Một số ví dụ về hằng số cụ thể là số như 5, 1.23, hoặc một chuỗi như ‘This is a string’ hay ‘It’s a string,’. Nó được gọi là “cụ thể” (literal) vì bạn đang sử dụng đúng nghĩa của nó. Số 2 luôn là 2 và không là gì khác - nó là một hằng số, vì giá trị của nó không thể thay đổi. Tất cả những thứ này cũng được gọi là hằng số cụ thể.

CÁC CON SỐ (NUMBERS)

Hai loại số thường được sử dụng là Số nguyên (Integers) và Số thực (floats). Một ví dụ là 2, đây chỉ là một con số. Ví dụ về số thực là 3.23 và 52.3E-4 (hay gọi tắt là floats). Ký hiệu E biểu thị cho lũy thừa 10. Trong trường hợp này, 52.3E-4 là 52.3×10^{-4} .

CHUỖI (STRINGS)

Chuỗi (string) là một tập hợp các ký tự. Về cơ bản, chuỗi chỉ là một loạt các từ. Gần như mọi chương trình Python bạn viết đều sẽ sử dụng chuỗi, vì vậy hãy rất cẩn thận với phần sau đây.

DẤU NHÁY ĐƠN (SINGLE QUOTE) Bạn có thể chỉ định chuỗi bằng cách sử dụng dấu nháy đơn, chẳng hạn như 'Quote me on this'. Tất cả các khoảng trắng, tức là dấu cách và tab, bên trong dấu ngoặc kép, đều được giữ nguyên.

DẤU NHÁY KÉP (DOUBLE QUOTE) Chuỗi trong dấu nháy kép hoạt động giống hệt như chuỗi trong dấu nháy đơn. Một ví dụ là "What's your name?".

DẤU NHÁY BA (TRIPLE QUOTE) Bạn có thể chỉ định chuỗi nhiều dòng bằng cách sử dụng dấu nháy ba - (""" hoặc '''). Bạn có thể tự do sử dụng dấu nháy đơn và dấu nháy kép bên trong dấu nháy ba. Một ví dụ là:

```
'''This is a multi-line string. This is the first line.
This is the second line.
"What's your name?," I asked.
He said "Bond, James Bond."
'''
```

Chuỗi là BẤT BIẾN (IMMUTABLE): Điều này có nghĩa là bạn không thể thay đổi một chuỗi sau khi đã tạo ra nó. Và ngay cả khi điều đó có vẻ là một điều tiêu cực, nó không hẳn là vậy. Chúng ta sẽ thấy các dịch vụ khác nhau mà chúng ta thấy sau này không bị giới hạn bởi điều này.

BIẾN (VARIABLES):

Chỉ sử dụng hằng số cụ thể sẽ sớm trở nên lặp đi lặp lại nếu đó là cách duy nhất để lưu trữ và thao tác dữ liệu. Đây là lúc các biến (variables) xuất hiện. Biến đúng như tên gọi của nó - giá trị của chúng có thể thay đổi, tức là, bạn có thể lưu trữ bất cứ thứ gì bằng một biến. Biến chỉ là một phần bộ nhớ của máy tính nơi thông tin được lưu trữ. Bạn cần một cách để kiểm soát các biến này và sau đó gọi chúng, trái ngược với các hằng số cụ thể.

TÊN ĐỊNH DANH (IDENTIFIERS)

Biến là ví dụ về tên định danh. Tên định danh (Identifiers) là tên được đặt để xác định một cái gì đó.

Có một số quy tắc bạn phải tuân theo khi đặt tên định danh:

- Ký tự đầu tiên của tên định danh phải là một chữ cái (ASCII viết hoa hoặc ASCII viết thường hoặc ký tự Unicode) hoặc dấu gạch dưới (_).
- Phần còn lại của tên định danh có thể bao gồm các chữ cái (ASCII viết hoa hoặc ASCII viết thường hoặc ký tự Unicode), dấu gạch dưới (_) hoặc chữ số (0-9).
- Tên định danh có phân biệt chữ hoa chữ thường. Ví dụ: myname và myName là không giống nhau. Lưu ý chữ n viết thường ở tên trước và chữ N viết hoa ở tên sau.
- Ví dụ về tên định danh hợp lệ là i, name_2_3. Ví dụ về tên định danh không hợp lệ là 2things, this is spaced out, my-name và >alb2_c3.

Kiểu dữ liệu (DATA TYPES)

Các biến có thể chứa các giá trị thuộc nhiều loại khác nhau, được gọi là kiểu dữ liệu. Các dạng chính, mà chúng ta đã mô tả, là số và chuỗi. Chúng ta sẽ thảo luận trong các chương sau về cách hình thành các lớp (class) của riêng mình.

Python coi mọi thứ được sử dụng trong chương trình như một thực thể. Điều này được nói một cách chung chung. Họ nói “đối tượng” (object), thay vì “một cái gì đó”. Python rất hướng đối tượng, theo nghĩa mọi thứ đều là một thực thể, bao gồm cả số, chuỗi và hàm.

Bây giờ chúng ta sẽ xem các biến có thể được sử dụng với các hằng số cụ thể như thế nào. Chạy chương trình và lưu ví dụ sau.

Cách viết một chương trình trong python:

Các bước cần tuân theo:

- Mở trình chỉnh sửa/Trình thông dịch (editor/Interpreter)
- Gõ mã chương trình như trong ví dụ
- Lưu nó dưới dạng tệp có tên với [DẤU CHẤM]py hoặc phần mở rộng “.py”
- Chạy trình thông dịch bằng lệnh `python filename.py` để chạy chương trình hoặc bạn có thể sử dụng nút run trên trình chỉnh sửa để chạy chương trình.
- Lưu ý: Tên tệp (Filename) có thể là bất cứ thứ gì nhưng nó phải có phần mở rộng .py

Ví dụ: Gõ và chạy chương trình sau

```
>>> # Tên tệp: var.py
i = 5
print (i)
i = i + 1
print (i)
s = "This is a multi-line string.
This is the second line
Welcome to artificial intelligence."
print (s)
```

Kết quả (Output):

```
5
6
This is a multi-line string.
This is the second line
Welcome to artificial Intelligence.
```

Cách nó hoạt động (How It Works)

Đây là cách chương trình này hoạt động. Đầu tiên, chúng ta gán giá trị hằng số cụ thể 5 cho biến `i` bằng toán tử gán (`=`). Dòng này được gọi là một câu lệnh (statement) vì nó nêu rõ rằng cần phải làm gì đó và trong trường hợp này, chúng ta kết nối tên biến `i` với giá trị 5. Tiếp theo, chúng ta in giá trị của `i` bằng câu lệnh `print`, không ngạc nhiên, nó chỉ in giá trị của biến ra màn hình.

Sau đó, chúng ta cộng 1 vào giá trị được lưu trữ trong `i` và lưu nó trở lại. Sau đó, chúng ta in nó ra và như mong đợi, chúng ta nhận được giá trị 6.

Tương tự, chúng ta gán chuỗi cụ thể cho biến `s` và sau đó in nó ra.

THỤT LỀ (INDENTATION)

Khoảng trắng (Whitespace) trong Python là rất quan trọng. Khoảng trắng ở đầu dòng thực sự cần thiết. Đó là cái được gọi là thụt lề (indentation). Khoảng trắng hoặc tab ở đầu dòng logic (leading whitespace) được sử dụng để quyết định mức độ thụt lề của dòng logic đó, có tác dụng xác định việc nhóm các câu lệnh.

Điều đó có nghĩa là tất cả các câu lệnh nói chung sẽ có cùng một mức thụt lề. Tất cả các câu lệnh này được gọi là một *khối* (block). Trong các trang tiếp theo, chúng ta có thể thấy các ví dụ về tầm quan trọng của các khối.

Một điều bạn nên biết là việc đặt sai vị trí sẽ dẫn đến lỗi trong chương trình. Ví dụ:

```
>>> i = 5
# Lỗi bên dưới! Lưu ý có một dấu cách ở đầu dòng
print ('Value is', i)
print ('I repeat, the value is', i)
```

Bạn sẽ nhận được lỗi sau khi chạy chương trình này:

```
File "whitespace.py", line 3
    print ('Value is', i)
    ^
```

IndentationError: unexpected indent

Lưu ý ở đầu dòng thứ hai có một dấu cách. Lỗi của Python có nghĩa là cú pháp của chương trình không chính xác và chương trình chưa được viết đúng. Điều này đảm bảo cho bạn rằng các khối câu lệnh mới không thể được bắt đầu một cách ngẫu nhiên (ngoại trừ khối chính hiện có mà bạn đã sử dụng từ trước đến nay).

TOÁN TỬ & BIỂU THỨC (OPERATORS & EXPRESSIONS)

Phần lớn các câu lệnh bạn viết (các dòng logic) sẽ chứa các từ. Một biểu thức `2 + 3` là một ví dụ đơn giản. Một biểu thức có thể được chia thành các toán tử (operators) và toán hạng (operands).

Toán tử là các hàm có thể làm điều gì đó, các ký hiệu như + hoặc các phím đặc biệt là biểu tượng của chúng. Toán tử cần dữ liệu khác và những dữ liệu đó được gọi là toán hạng. 2 và 3 là các toán hạng trong trường hợp này.

Chúng ta sẽ thảo luận ngắn gọn về các toán tử và cách sử dụng chúng. Lưu ý rằng các biểu thức trong ví dụ có thể được kiểm tra tương tác với trình thông dịch. Ví dụ, dấu nhắc trình thông dịch Python tương tác được sử dụng để kiểm tra biểu thức $7 + 5$:

```
>>>7 + 5
12
>>>3 * 8
24
>>>
```

Dưới đây là bản tóm tắt ngắn gọn về các toán tử có sẵn:

- **+** (**cộng**) Cộng hai đối tượng. $+ 5$ cho kết quả 8. 'a' + 'b' cho kết quả 'ab'.
- **-** (**trừ**) Thực hiện phép trừ một số cho số khác; nếu toán hạng đầu tiên không có, nó được giả định là số không. -5.2 cho một số âm và $50 - 24$ cho kết quả 26.
- ***** (**nhân**) Thực hiện phép nhân hai số hoặc trả về chuỗi được lặp lại nhiều lần. $* 3$ cho kết quả 6. 'la' * 3 cho kết quả 'lalala'.
- ****** (**lũy thừa**) Trả về x mũ y. $** 4$ cho kết quả 81 (tức là $3 * 3 * 3 * 3$). (Lưu ý: tài liệu gốc ghi $3 * 3 * 3 * 3 * 3$ là sai, phải là 4 lần 3)
- **/** (**chia**) Chia x cho y. 3 cho kết quả 4.3333333333333333.
- **//** (**chia và làm tròn xuống**) Chia x cho y và làm tròn câu trả lời xuống giá trị số nguyên gần nhất. Lưu ý rằng nếu một trong các giá trị là số thực (float), bạn sẽ nhận lại một số thực. $13 // 3$ cho kết quả 4.
- [Tiếp tục danh sách toán tử...]
- $-13 // 3$ gives -5
- $9//1.81$ gives 4.0
- **%** (**modulo**)
 - Trả về phần dư của phép chia.
 - 3 cho kết quả 1. $-25.5\% 2.25$ cho kết quả 1.5.
- **<<** (**dịch trái**)
 - Dịch các bit của số sang bên trái theo số lượng bit được chỉ định. (Mỗi số được biểu diễn trong bộ nhớ bằng các bit hoặc chữ số nhị phân, tức là 0 và 1)
 - $2 << 2$ cho kết quả 8. 2 được biểu diễn bằng 10 ở dạng bit. Dịch trái 2 bit sẽ cho 1000, đại diện cho số 8 trong hệ thập phân.
- **>>** (**dịch phải**)
 - Dịch các bit của số sang bên phải theo số lượng bit được chỉ định.

- $11 \gg 1$ cho kết quả 5. 11 được biểu diễn bằng bit là 1011, khi dịch phải 1 bit sẽ cho kết quả là 101, tức là số 5 trong hệ thập phân.
- **& (AND bit-wise)**
 - Phép toán AND bit-wise của các số.
 - $5 \& 3$ cho kết quả 1.
- (OR bit-wise)
 - Phép toán OR bit-wise của các số.
 - $5 | 3$ cho kết quả 7.
- **^ (XOR bit-wise)**
 - Phép toán XOR bit-wise của các số.
 - $^ 3$ cho kết quả 6.
- **~ (đảo bit-wise)**
 - Phép đảo bit-wise của x là $-(x + 1)$.
 - ~ 5 cho kết quả -6.
- **< (nhỏ hơn)**
 - Trả về liệu x có nhỏ hơn y hay không. Tất cả các toán tử so sánh trả về True hoặc False. Lưu ý cách viết hoa của các tên này.
 - $5 < 3$ cho kết quả False và $3 < 5$ cho kết quả True. Các phép so sánh có thể được nối tùy ý: $3 < 5 < 7$ cho kết quả True.
- **> (lớn hơn)**
 - Trả về liệu x có lớn hơn y hay không.
 - $5 > 3$ trả về True. Nếu cả hai toán hạng là số, chúng sẽ được chuyển đổi sang một kiểu chung. Nếu không, nó luôn trả về False.
- **<= (nhỏ hơn hoặc bằng)**
 - Trả về liệu x có nhỏ hơn hoặc bằng y hay không.
 - $x=3; y=6; x \leq y$ trả về True.
- **>= (lớn hơn hoặc bằng)**
 - Trả về liệu x có lớn hơn hoặc bằng y hay không.
 - $x=4; y=3; x \geq 3$ trả về True.
- **== (bằng)**
 - So sánh xem các đối tượng có bằng nhau không.
 - $x=2; y=2; x==y$ trả về True. (Tài liệu gốc ghi $x = y$)
 - $x='str'; y=\textcircled{str}^{\text{prime}}; x==y$ trả về False. (Tài liệu gốc ghi $x = y$)
 - $x='str'; y=\textcircled{str}^{\text{prime}}; x==y$ trả về True. (Tài liệu gốc ghi $x = y$)
- **!= (không bằng)**
 - So sánh xem các đối tượng có không bằng nhau không.
 - $x=2; y=3; x!=y$ trả về True.

- **not (phủ định logic NOT)**
 - Nếu x là True, nó trả về False. Nếu x là False, nó trả về True.
 - `x= True; not x` trả về False.
- **and (VÀ logic AND)**
 - `x and y` trả về False nếu x là False, ngược lại nó trả về giá trị của y.
 - `x= False; y= True; x and y` trả về False vì x là False. Trong trường hợp này, Python sẽ không đánh giá y vì nó biết rằng về bên trái của biểu thức 'and' là False, điều này ngụ ý rằng toàn bộ biểu thức sẽ là False bất kể các giá trị khác. Điều này được gọi là **đánh giá ngắn mạch** (short-circuit evaluation).
- **or (HOẶC logic OR)**
 - Nếu x là True, nó trả về True, ngược lại nó trả về giá trị của y.
 - `x= True; y= False; x or y` trả về True. Đánh giá ngắn mạch cũng áp dụng ở đây.

LUỒNG ĐIỀU KHIỂN (CONTROL FLOW)

Trong các chương trình chúng ta đã thấy cho đến nay, các câu lệnh từ trên xuống dưới của Python cũng đã được thực hiện một cách trung thực. Sẽ thế nào nếu nhịp điệu hoạt động của nó thay đổi? Ví dụ, bạn có muốn hệ thống đưa ra một số quyết định và làm những việc nhất định tùy theo các tình huống khác nhau, như in 'Good morning' (Chào buổi sáng) hoặc 'Good evening' (Chào buổi tối) tùy thuộc vào thời điểm trong ngày không?

Như bạn có thể tưởng tượng, các câu lệnh luồng điều khiển được sử dụng để thực hiện điều này. Trong Python có ba câu lệnh luồng điều khiển—**if**, **for** và **while**.

- **Câu lệnh If (If Statement)** Nếu điều kiện là đúng, chúng ta chạy một khối (được gọi là **khối if**), ngược lại chúng ta phải xử lý một khối (được gọi là **khối else**). (Câu lặp lại trong tài liệu gốc) Phần else là tùy chọn.

Ví dụ (lưu với tên if.py):

```
>>>number = 77
guess = int(input('Enter an integer : ')) # Nhập một số nguyên :
if guess == number:
    # New block starts here # Khối mới bắt đầu ở đây
    print('Congratulations, you guessed it.') # Chúc mừng, bạn đã đoán đúng.
    print(' (but you do not win any prizes!)') # (nhưng bạn không thắng giải thưởng nào!)
    # New block ends here # Khối mới kết thúc ở đây
elif guess < number:
    # Another block, in this we can use if for n number of times.
    # Khối khác, trong khối này chúng ta có thể sử dụng if cho n số lần.
    print('No, it is a little higher than that') # Không, nó cao hơn một chút
```

```
# You can do whatever you want in a block... # Bạn có thể làm bất cứ điều gì bạn muốn trong một khối...
else:
    print ('No, it is a little lower than that') # Không, nó thấp hơn một chút
    # you must have guessed > number to reach here # bạn phải đoán > số để đến đây
    print ('Done') # Đã xong
# This last statement is always executed, # Câu lệnh cuối cùng này luôn được thực thi,
# after the if statement is executed.: # sau khi câu lệnh if được thực thi.:
```

KẾT QUẢ:

```
python if.py
Enter an integer: 22
No, it is a little higher than that
Done
```

Chúng ta lấy dự đoán từ người dùng trong chương trình này và kiểm tra xem nó có phải là số chúng ta có hay không. Chúng ta đã chỉ định số của biến cho bất kỳ số nguyên nào, giả sử là 23. (Lưu ý: mã nguồn ghi 77) Sau đó, chúng ta sử dụng hàm input để lấy dự đoán của người dùng. ...là các đoạn chương trình có thể tái sử dụng. Chúng ta cung cấp một chuỗi cho hàm input tích hợp để in ra màn hình và đợi người dùng nhập.

Hàm input() trả về những gì chúng ta đã nhập dưới dạng một chuỗi sau khi chúng ta gõ gì đó và nhấn phím kbd: [enter]. Chuỗi này được dịch sang một số nguyên bằng int và được lưu trữ trong biến guess (dự đoán). (Tài liệu gốc ghi “conjecture”) Nói chung, int là một lớp (class), nhưng tất cả những gì bạn cần biết ngay bây giờ là bạn có thể sử dụng nó để chuyển đổi một chuỗi thành một số nguyên.

Tiếp theo, chúng ta so sánh dự đoán của người dùng với số chúng ta đã chọn. bằng cách in một thông báo thành công nếu chúng bằng nhau. Lưu ý, để cho Python biết câu lệnh nào thuộc khối nào, chúng ta sử dụng các mức thụt lề. Do đó, thụt lề trong Python rất cần thiết.

Lưu ý câu lệnh if ở cuối chứa một dấu hai chấm: chúng ta thông báo cho Python rằng sau nó là một khối các câu lệnh.

Sau đó, chúng ta kiểm tra xem giả định (guess) có nhỏ hơn con số không, và nếu có, chúng ta sẽ nói cho người tiêu dùng biết rằng con số cần đoán cao hơn một chút so với giả định. Mệnh đề elif mà chúng ta đã sử dụng ở đây đơn giản là kết hợp hai câu lệnh if-else tương tự thành một câu lệnh if-elif-else duy nhất. Nó cải thiện hệ thống và giảm mức độ thụt lề cần thiết.

Các câu lệnh elif và else phải luôn có dấu hai chấm ở cuối phần logic, theo sau là một chuỗi các câu lệnh (tất nhiên là được thụt lề đúng cách).

Bạn có thể có một câu lệnh if khác bên trong khối if của một câu lệnh if và cứ như thế—đây được gọi là một câu lệnh if lồng nhau (nested if).

Hãy nhớ rằng các phần elif và else là tùy chọn. Một câu lệnh if hợp lệ tối thiểu là:

```
>>>if True:
    print ('Yes, it is true') # Vâng, điều đó là đúng
```

Sau khi Python hoàn thành toàn bộ câu lệnh if cùng với các mệnh đề elif và else tương ứng, nó sẽ chuyển sang câu lệnh tiếp theo trong chuỗi sau câu lệnh if. Câu lệnh tiếp theo là câu lệnh print ('Done'). Đây là một khối lớn (nơi chương trình bắt đầu thực thi). Sau đó, Python thấy phần cuối của chương trình và kết thúc.

- **Câu lệnh For (For Statement)**

Câu lệnh for..in là một câu lệnh lặp (looping statement - *Lưu ý: đây có thể là lỗi in ấn, “looping statement” mới là thuật ngữ đúng*), nó duyệt qua từng phần tử trong một chuỗi (sequence). Chúng ta sẽ tìm hiểu thêm về chuỗi trong các phần sau. Bây giờ bạn chỉ cần biết rằng chuỗi chỉ là một dãy các đối tượng.

Ví dụ (lưu với tên for.py):

```
for i in range (1, 5):
    print (i)
else:
    print ('The for loop is over') # Vòng lặp for đã kết thúc
```

Kết quả:

```
python for.py
1
2
3
4
The for loop is over
```

Chúng ta in một chuỗi số trong chương trình này. Chúng ta sử dụng hàm range tích hợp để tạo ra chuỗi số này. Tất cả những gì chúng ta làm ở đây là cung cấp cho nó hai số và nó trả về một chuỗi số từ số thứ nhất đến số thứ hai. Ví dụ, range(1, 5) cung cấp chuỗi [1, 2, 3, 4].

Theo mặc định, range có bước nhảy (move number) là 1. Nếu chúng ta cung cấp số thứ ba, nó sẽ là bước nhảy trong chuỗi. Ví dụ, range(1, 5, 2).....liệt kê [1, 3]. Lưu ý rằng range mở rộng *cho đến* số thứ hai, tức là, số thứ hai không được bao gồm.

Hãy nhớ rằng, nếu bạn muốn có một bộ số hoàn chỉnh, ví dụ, list(range(5)) sẽ cho kết quả [0, 1, 2, 3, 4] (list() được giải thích trong chương cấu trúc dữ liệu). Vòng lặp for sau đó lặp qua danh sách này (list(s))—ví dụ, for i in range(1, 5) tương đương với việc gán từng

số trong chuỗi [1, 2, 3, 4] cho i. Trong trường hợp này, giá trị chỉ được in ra trong câu lệnh.

Lưu ý rằng phần else là tùy chọn. Khi được sử dụng, nó chỉ được thực hiện sau khi vòng lặp kết thúc, trừ khi gặp câu lệnh break.

Lưu ý rằng vòng lặp for..in hoạt động với mọi chuỗi. Ở đây, chúng ta có một danh sách các số được tạo bởi hàm range tích hợp, nhưng chúng ta có thể sử dụng bất kỳ loại chuỗi đối tượng nào!.

- **Câu lệnh While (While Statement)**

Câu lệnh while cho phép bạn thực thi lặp đi lặp lại một chuỗi các câu lệnh, miễn là một điều kiện còn đúng (valid). Đây là một ví dụ về cái gọi là câu lệnh lặp (looping statement). Một câu lệnh while có thể bao gồm một mệnh đề else tùy chọn.

Ví dụ (lưu với tên while.py):

```
number = 23
running = True
while running:
    guess = int(input('Enter an integer: ')) # Nhập một số nguyên:

    if guess == number:
        print('Congratulations, you guessed it.') # Chúc mừng, bạn đã đoán
        đúng.
        running = False # điều này khiến vòng lặp while dừng lại
    elif guess < number:
        print('No, it is a little higher than that.') # Không, nó cao hơn
        một chút.
    else:
        print('No, it is a little lower than that.') # Không, nó thấp hơn
        một chút.
    else:
        print('The while loop is over.') # Vòng lặp while đã kết thúc.
# Do anything else you want to do here # Làm bất cứ điều gì khác bạn muốn làm
ở đây
print('Done') # Đã xong
```

Kết quả:

```
python while.py
Enter an integer: 50
No, it is a little lower than that.
Enter an integer: 22
No, it is a little higher than that.
Enter an integer: 23
```


Congratulations, you guessed it.
The while loop is over.
Done

Chúng ta vẫn chơi trò chơi đoán số trong chương trình này, nhưng lợi ích là người dùng có thể đoán cho đến khi họ nghĩ ra số đúng, không giống như phần trước—không cần phải chạy nó lặp đi lặp lại. Điều này cho thấy việc sử dụng Câu lệnh While một cách hợp lý.

Chúng ta chuyển phần nhập liệu vào bên trong vòng lặp while và đặt biến running thành True trước vòng lặp while. Đầu tiên, chúng ta kiểm tra xem biến running có phải là True không và sau đó tiếp tục thực thi (các câu lệnh bên trong). Sau khi khối này được hoàn thành, điều kiện (biến running) được kiểm tra lại. Nếu nó vẫn đúng (valid), chúng ta sẽ chạy lại khối While, nếu không, khối else tùy chọn sẽ tiếp tục được thực thi, và sau đó câu lệnh tiếp theo sẽ được thực thi.

Khối else được thực thi khi điều kiện while trở thành không đúng —ngay cả khi điều kiện được kiểm tra là sai ngay từ lần đầu tiên. Nếu có một mệnh đề else cho vòng lặp while, nó sẽ luôn hoạt động trừ khi bạn thoát ra khỏi vòng lặp bằng break.

True và False được gọi là các dạng Boolean (Boolean forms) và có thể được coi là tương đương với 1 và 0.

CÂU LỆNH BREAK (THE BREAK STATEMENT)

Câu lệnh break được sử dụng để thoát ra khỏi một câu lệnh lặp, tức là, dừng một câu lệnh lặp, ngay cả khi điều kiện của nó chưa trở thành False hoặc chuỗi các mục chưa được duyệt qua hoàn toàn.

Một điểm quan trọng là bất kỳ khối else nào đi kèm của vòng lặp sẽ không được thực thi khi bạn thoát ra khỏi vòng lặp for hoặc while bằng break.

Ví dụ (lưu với tên break.py):

```
while True:
    s = input('Enter something : ') # Nhập gì đó :
    if s == 'quit':
        break
    print('Length of the string is', len(s)) # Độ dài của chuỗi là
print('Done') # Đã xong
```

Kết quả:

```
python break.py
Enter something : Programming is fun
Length of the string is 18
Enter something : When the work is done
```

Length of the string is 21
Enter something : if you wanna make your work also fun:

Length of the string is 37
Enter something : use Python!
Length of the string is 11
Enter something : quit
Done

CÂU LỆNH CONTINUE (THE CONTINUE STATEMENT)

Sử dụng câu lệnh continue, Python sẽ bỏ qua phần còn lại của các câu lệnh trong khối vòng lặp hiện tại và bắt đầu với vòng lặp (iteration) tiếp theo của vòng lặp.

Ví dụ (lưu với tên continue.py):

```
while True:
    s = input('Enter something : ') # Nhập gì đó :
    if s == 'quit':
        break
    if len(s) < 3:
        print('Too small') # Quá nhỏ
        continue
    print('Input is of sufficient length') # Dữ liệu nhập có độ dài đủ
# Do other kinds of processing here... # Thực hiện các loại xử lý khác ở đây...
```

Kết quả:

```
python continue.py
Enter something : a
Too small
Enter something : 12
Too small
Enter something : abc
Input is of sufficient length
Enter something : quit
```

HÀM (FUNCTIONS)

Hàm là những đoạn chương trình có thể tái sử dụng. Bạn có thể đặt tên cho một khối các câu lệnh, và bạn có thể sử dụng tên đó ở bất kỳ đâu và bất kỳ số lần nào để chạy khối lệnh đó trong chương trình của mình. Đây được gọi là *gọi hàm* (function calling). Chúng ta đã sử dụng một số hàm tích hợp như len và range.

Có thể đây là khối xây dựng chính cho bất kỳ ứng dụng không tầm thường nào (trong bất kỳ ngôn ngữ lập trình nào).

Các hàm được định nghĩa bằng từ khóa def. Sau từ khóa này là tên hàm, theo sau là một cặp dấu ngoặc đơn có thể chứa tên của các biến...và dấu hai chấm cuối cùng kết thúc dòng. Tiếp theo là tập hợp các khai báo (declarations) thuộc về hàm này.

Một ví dụ cho thấy điều này thực sự đơn giản:

Ví dụ (lưu với tên function1.py):

```
def say_hello ():  
    # block belonging to the function # khối lệnh thuộc về hàm  
    print ('Welcome to agriculture') # Chào mừng đến với nông nghiệp  
# End of function # Kết thúc hàm  
  
say_hello () # call the function # gọi hàm  
say_hello () # call the function again # gọi lại hàm
```

Kết quả:

```
python function1.py  
Welcome to agriculture  
Welcome to agriculture
```

BIẾN CỤC BỘ (LOCAL VARIABLES)

Khi bạn khai báo các biến bên trong một định nghĩa hàm, chúng không liên quan gì đến các biến khác có cùng tên được sử dụng bên ngoài hàm - tức là, tên biến là cục bộ (local) đối với hàm. Đây được gọi là *phạm vi* (scope) của biến. Tất cả các biến có phạm vi là khối mà chúng được khai báo, bắt đầu từ điểm định nghĩa của tên.

Ví dụ (lưu với tên function_local.py):

```
x = 50  
  
def func (x):  
    print ('x is', x) # x là  
    x = 2  
    print ('Changed local x to', x) # Đã thay đổi x cục bộ thành  
  
func (x)  
print ('x is still', x) # x vẫn là
```

(Lưu ý: Có một lỗi logic trong mã nguồn gốc . func(x) được gọi trước khi x bên trong hàm được thay đổi. Bản dịch vẫn giữ nguyên mã, nhưng kết quả đầu ra sẽ phản ánh mã đã cho.)

Kết quả:

```
python function_local.py  
x is 50
```

Changed local x to 2
x is still 50

CÂU LỆNH GLOBAL (THE GLOBAL STATEMENT)

Nếu bạn muốn gán một ý nghĩa (giá trị) cho một tên biến ở cấp cao nhất của chương trình (tức là, không nằm trong bất kỳ phạm vi nào, chẳng hạn như hàm hoặc lớp), bạn phải nói cho Python biết rằng một tên nhất định không phải là cục bộ, mà là toàn cục (global) . Chúng ta sử dụng câu lệnh global để làm điều này. Nếu không có câu lệnh global, bạn không thể gán giá trị cho một biến được định nghĩa bên ngoài một hàm.

Bạn có thể sử dụng các giá trị được định nghĩa bên ngoài hàm (mà không cần global nếu chỉ đọc). Điều này không được khuyến khích và nên tránh vì người dùng chương trình không biết ý nghĩa của biến này ở đâu. Từ câu lệnh global, có thể thấy rõ ràng rằng biến được định nghĩa trong khối bên ngoài.

Ví dụ (lưu với tên `function_global.py`):

```
x = 50
def func ():
    global x
    print ('x is', x) # x là
    x = 2
    print ('Changed global x to', x) # Đã thay đổi x toàn cục thành

func ()
print ('Value of x is', x) # Giá trị của x là
```

(Lưu ý: Có một lỗi nhỏ trong mã nguồn gốc, `print ('Changed global x to', )` thiếu biến `x`. Tôi đã sửa lại dựa trên ngữ cảnh là `print ('Changed global x to', x)` để khớp với kết quả đầu ra.)

Kết quả:

```
python function_global.py
x is 50
Changed global x to 2
Value of x is 2
```

GIÁ TRỊ THAM SỐ MẶC ĐỊNH (DEFAULT ARGUMENT VALUES)

Đối với một số hàm, bạn có thể tùy chọn hóa các tham số và sử dụng các giá trị mặc định nếu người dùng không muốn cung cấp giá trị cho chúng. Các giá trị tham số mặc định được sử dụng để làm điều đó. Bằng cách thêm toán tử gán (=) vào tên tham số trong đặc tả tham số, bạn có thể đặt giá trị tham số mặc định cho các tham số.

Hãy nhớ rằng câu lệnh mặc định là một giá trị không đổi. Cụ thể hơn—nó sẽ được thảo luận chi tiết trong các chương sau—các giá trị tham số mặc định phải là bất biến (immutable).

Ví dụ (lưu với tên `function_default.py`):

```
def say (message, times = 1):  
    print (message * times)  
  
say ('Hello')  
say ('World', 5)
```

(Lưu ý: Mã nguồn gốc có một dấu * lạc. Dựa trên ngữ cảnh, nó có thể là lỗi định dạng. Câu lệnh đúng phải là `print (message * times)`.)

Kết quả:

```
python function_default.py  
Hello  
WorldWorldWorldWorldWorld
```

THAM SỐ TỪ KHÓA (KEYWORD ARGUMENTS)

Cú pháp đặc biệt `**kwargs` trong định nghĩa hàm trong python được sử dụng để truyền một danh sách tham số có độ dài thay đổi, có từ khóa. Chúng ta sử dụng tên `kwargs` với dấu sao kép. Lý do là vì dấu sao kép cho phép chúng ta truyền qua các tham số từ khóa (và bất kỳ số lượng nào trong số chúng).

Tham số từ khóa (keyword argument) là nơi bạn cung cấp tên cho biến khi bạn truyền nó vào hàm. Người ta có thể nghĩ `kwargs` như một từ điển (dictionary) ánh xạ mỗi từ khóa tới giá trị mà chúng ta truyền cùng với nó. Đó là lý do tại sao khi chúng ta lập qua `kwargs`, dường như không có thứ tự nào mà chúng được in ra.

Ví dụ (lưu với tên `function_keyword.py`):

```
def func (a, b=5, c=10):  
    print ('a is', a, 'and b is', b, 'and c is', c) # a là, và b là, và c là  
  
func (3, 7)  
func (25, c=24)  
func (c=50, a=100)
```

Kết quả:

```
python function_keyword.py  
a is 3 and b is 7 and c is 10  
a is 25 and b is 5 and c is 24  
a is 100 and b is 5 and c is 50
```

THAM SỐ VARARGS (VARARGS PARAMETERS)

Đôi khi bạn có thể muốn định nghĩa một hàm có thể nhận bất kỳ số lượng tham số nào, tức là, số lượng đối số thay đổi (variable number of arguments), điều này có thể đạt được bằng cách sử dụng các dấu sao (lưu với tên `function_varargs.py`):

```
def total (a=5, *numbers, **phonebook):  
    print ('a', a)  
  
    #iterate through all the items in tuple # lặp qua tất cả các mục trong  
tuple  
    for single_item in numbers:  
        print ('single_item', single_item)  
  
    #iterate through all the items in dictionary # lặp qua tất cả các mục  
trong từ điển  
    for first_part, second_part in phonebook.items ():  
        print (first_part, second_part)  
  
total (10, 1, 2, 3, Jack=1123, John=2231, Inge=1560)
```

Kết quả:

```
python function_varargs.py  
a 10
```

```
single_item 1  
single_item 2  
single_item 3  
Inge 1560  
John 2231  
Jack 1123
```

CÂU LỆNH RETURN (RETURN STATEMENT)

Câu lệnh `return` được sử dụng để trả về (kết quả) từ một hàm, tức là, thoát ra khỏi hàm. Chúng ta cũng có thể tùy chọn trả về một giá trị từ hàm.

Ví dụ (lưu với tên `function_return.py`):

```
def maximum (x, y):  
    if x > y:  
        return x  
    elif x == y:  
        return 'The numbers are equal' # Các số bằng nhau  
    else:  
        return y  
  
print (maximum (2, 3))
```

Kết quả:

python function_return.py
3

CẤU TRÚC DỮ LIỆU (DATA STRUCTURES)

Về cơ bản, cấu trúc dữ liệu chính là như vậy—các cấu trúc có thể giữ tất cả dữ liệu lại với nhau. Nói cách khác, chúng dùng để xử lý một danh sách các thông tin tương tự. Trong Python có bốn cấu trúc dữ liệu tích hợp sẵn là **list** (danh sách), **tuple** (bộ), **dictionary** (từ điển) và **set** (tập hợp). Chúng ta sẽ xem cách sử dụng từng loại và cách chúng cải thiện cuộc sống của chúng ta.

LIST (DANH SÁCH): List là cấu trúc dữ liệu bao gồm một tập hợp các đối tượng có thứ tự, tức là, một chuỗi các đối tượng có thể được lưu trữ trong một danh sách. Bạn có thể dễ dàng hình dung điều này, nếu bạn có thể nghĩ về một danh sách mua sắm với một danh sách những thứ cần mua, ngoại trừ việc mỗi mục thực sự được đặt riêng trên danh sách mua sắm của bạn, trong khi danh sách Python dùng dấu phẩy.

Tập hợp các mục (items) phải được bao gồm trong dấu ngoặc vuông, để Python có thể hiểu rằng bạn đang định nghĩa một tập hợp (collection). Một khi bạn đã xây dựng một danh sách, bạn có thể thêm, xóa hoặc kiểm tra các mục trong danh sách. Vì chúng ta có thể thêm và xóa các mục, danh sách là một kiểu dữ liệu *khả biến* (mutable), tức là, nó có thể được sửa đổi.

Ví dụ (lưu với tên list.py):

```
# This is my shopping list # Đây là danh sách mua sắm của tôi
shoplist = ['apple', 'mango', 'carrot', 'banana'] # táo, xoài, cà rốt, chuối

print('I have', len(shoplist), 'items to purchase.') # Tôi có ... món đồ
cần mua.

print('These items are:', end=' ') # Những món đồ đó là:
for item in shoplist:
    print(item, end=' ')

print('\nI also have to buy rice.') # Tôi cũng phải mua gạo.
shoplist.append('rice') # thêm 'gạo'
print('My shopping list is now', shoplist) # Danh sách mua sắm của tôi bây
giờ là

print('I will sort my list now') # Bây giờ tôi sẽ sắp xếp danh sách của mình
shoplist.sort()
print('Sorted shopping list is', shoplist) # Danh sách mua sắm đã sắp xếp là

print('The first item I will buy is', shoplist[0]) # Món đồ đầu tiên tôi sẽ
```

```

mua là
olditem = shoplis[t]
del shoplis[t]
print ('I bought the', olditem) # Tôi đã mua
print ('My shopping list is now', shoplis) # Danh sách mua sắm của tôi bây giờ
là

```

Kết quả:

```

python ds_using_list.py
I have 4 items to purchase.
These items are: apple mango carrot banana
I also have to buy rice.
My shopping list is now ['apple', 'mango', 'carrot', 'banana', 'rice']
I will sort my list now
Sorted shopping list is ['apple', 'banana', 'carrot', 'mango', 'rice']
The first item I will buy is apple
I bought the apple
My shopping list is now ['banana', 'carrot', 'mango', 'rice']

```

- **TUPLE (BỘ):** Tuples (Bộ) được sử dụng để giữ nhiều đối tượng cùng nhau. Hãy coi chúng như danh sách (lists), nhưng không có các tính năng mở rộng mà bạn nhận được từ lớp (class) danh sách. Một trong những đặc điểm quan trọng nhất của tuples là tính *bất biến* (unchangeable) của chúng, nghĩa là, bạn không thể thay đổi tuples. Tuples được xác định bằng cách chỉ định các đối tượng được phân tách bằng dấu phẩy, với dấu ngoặc đơn tùy chọn. Tuples thường được sử dụng trong trường hợp một câu lệnh hoặc hàm do người dùng định nghĩa có thể giả định một cách an toàn rằng tập hợp các giá trị không thay đổi. **Ví dụ (lưu với tên tuple.py):**

```

# I would recommend always using parentheses
# to indicate start and end of tuple
# even though parentheses are optional.
# Explicit is better than implicit.
# Tôi khuyên bạn nên luôn sử dụng dấu ngoặc đơn
# để chỉ ra điểm bắt đầu và kết thúc của tuple
# mặc dù dấu ngoặc đơn là tùy chọn.
# Rõ ràng (Explicit) tốt hơn là ngầm định (implicit).
zoo = ('python', 'elephant', 'penguin')
print ('Number of animals in the zoo is', len (zoo)) # Số lượng động
vật trong sở thú là
new_zoo = 'monkey', 'camel', zoo # parentheses not required but are a
good idea # không yêu cầu dấu ngoặc đơn nhưng là một ý hay
print ('Number of cages in the new zoo is', len (new_zoo)) # Số chuồng
trong sở thú mới là
print ('All animals in new zoo are', new_zoo) # Tất cả động vật trong
sở thú mới là

```



```
print ('Animals brought from old zoo are', new_zoo [2]) # Động vật mang
từ sở thú cũ là
print ('Last animal brought from old zoo is', new_zoo [2] [2]) # Động
vật cuối cùng mang từ sở thú cũ là
print ('Number of animals in the new zoo is', len (new_zoo)-1+len
(new_zoo [2])) # Số lượng động vật trong sở thú mới là
```

(Lưu ý: Có vẻ như mã nguồn bị cắt xén và thiếu/lỗi. Ví dụ, new_zoo và zoo đều được coi là “chuồng” (cages) trong len(new_zoo) và “động vật” (animals) trong len(zoo). Phép tính số lượng động vật cuối cùng cũng có vẻ không chính xác. Tôi đã dịch nguyên trạng mã và kết quả.)

Kết quả:

```
python ds_using_tuple.py
Number of animals in the zoo is 3
Number of cages in the new zoo is 3
All animals in new zoo are ('monkey', 'camel', ('python', 'elephant', 'penguin'))
Animals brought from old zoo are ('python', 'elephant', 'penguin')
Last animal brought from old zoo is penguin
Number of animals in the new zoo is 5
```

- **DICTIONARY (TỪ ĐIỂN)** Từ điển (dictionary) giống như một cuốn sổ địa chỉ, trong đó bạn có thể tìm thấy địa chỉ hoặc thông tin liên lạc của một người chỉ bằng tên của người đó. Nghĩa là, nó liên kết **khóa** (keys) (tên) với **giá trị** (values) (chi tiết). Xin nhớ rằng khóa (key) phải là duy nhất, giống như nếu bạn có hai người cùng tên, bạn không thể tìm thấy các khóa chính xác. Bạn chỉ có thể sử dụng các đối tượng *bất biến* (immutable) (như chuỗi) cho khóa của từ điển, nhưng bạn có thể sử dụng các đối tượng *khả biến* (mutable - tài liệu gốc ghi “dynamic or moving”) hoặc bất biến cho các giá trị của từ điển. Điều này đơn giản có nghĩa là chỉ các đối tượng đơn giản mới có thể được sử dụng cho khóa. Các cặp khóa và giá trị được định nghĩa trong từ điển bằng ký hiệu d = {key1: value1, key2: value2}. Lưu ý rằng các cặp khóa-giá trị được phân tách bằng dấu hai chấm và các cặp được phân tách bằng dấu phẩy, tất cả được chứa trong một cặp dấu ngoặc nhọn. Lưu ý rằng các cặp khóa-giá trị không được sắp xếp theo bất kỳ cách nào trong từ điển. Nếu bạn muốn một thứ tự cụ thể, bạn phải tự sắp xếp nó trước khi sử dụng. Các từ điển bạn có thể sử dụng là các thực thể/đối tượng thuộc loại dict.

Ví dụ (lưu với tên dict.py):

```
# 'ab' is short for 'a'ddress'b'ook # 'ab' là viết tắt của 's'ổ 'đ'ịa'c'hỉ
ab = {
    'Swaroop': 'swaroop@swaroopch.com',
    'Larry': 'larry@wall.org',
    'Matsumoto': 'matz@ruby-lang.org',
    'Spammer': 'spammer@hotmail.com'
```

```

}

print ("Swaroop's address is", ab['Swaroop']) # Địa chỉ của Swaroop là

# Deleting a key-value pair # Xóa một cặp khóa-giá trị
del ab['Spammer']

print ('\nThere are {} contacts in the address-book\n'.format(len (ab))) # Có
... liên hệ trong sổ địa chỉ

for name, address in ab.items ():
    print ('Contact {} at {}'.format (name, address)) # Liên hệ ... tại ...

# Adding a key-value pair # Thêm một cặp khóa-giá trị
ab['Guido'] = 'guido@python.org'

if 'Guido' in ab:
    print ("\nGuido's address is", ab['Guido']) # Địa chỉ của Guido là

```

Kết quả:

```

python ds_using_dict.py
Swaroop's address is swaroop@swaroopch.com

```

There are 3 contacts in the address-book

```

Contact Swaroop at swaroop@swaroopch.com
Contact Matsumoto at matz@ruby-lang.org
Contact Larry at larry@wall.org

```

Guido's address is guido@python.org

- **SET (TẬP HỢP):** Sets (Tập hợp) là các mảng (arrays) không có thứ tự của các mục đơn giản. Chúng được sử dụng khi sự hiện diện của một mục trong một tập hợp quan trọng hơn là thứ tự hoặc số lượng của nó. Bạn có thể sử dụng các tập hợp để kiểm tra tư cách thành viên (membership), xem nó có phải là tập hợp con (subset) của một tập hợp khác hay không, tìm giao (crossing) giữa hai tập hợp, v.v.. **Ví dụ (chạy nó trong trình thông dịch python hoặc trong IDE):**

```

>>> bri = set (['brazil', 'russia', 'india'])
>>> 'india' in bri
True
>>> 'usa' in bri
False
>>> bric = bri.copy ()
>>> bric.add ('china')
>>> bric.issuperset (bri) # bric có phải là tập hợp cha của bri không

```

```
True
>>> bri.remove('russia')
>>> bri & bric # HOẶC bri.intersection(bric) (phép giao)
{'brazil', 'india'}
```

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG (OBJECT ORIENTED PROGRAMMING)

Trong tất cả các chương trình chúng ta đã viết cho đến nay, chúng ta đã xây dựng chương trình của mình để thao tác dữ liệu, tức là các khối câu lệnh. Đây được gọi là cách lập trình *hướng thủ tục* (process-oriented). Một cách khác để tổ chức chương trình của bạn là kết hợp dữ liệu và các tính năng (features) và gói chúng lại thành một thứ gọi là *thực thể* (entity).

Phong cách lập trình này được gọi là *hướng đối tượng* (object-oriented). Bạn thường có thể sử dụng lập trình hướng thủ tục, nhưng bạn có thể sử dụng các kỹ thuật lập trình hướng đối tượng khi viết các chương trình lớn hoặc nếu có một vấn đề phù hợp hơn với quy trình này.

Hai khía cạnh chính của lập trình hướng đối tượng là **lớp** (classes) và **đối tượng** (objects). Một lớp cung cấp một loại (kind) khác nhau, trong đó các đối tượng là các *thực thể* (instances) của lớp. Một sự tương tự là bạn có thể có các biến int, điều này cho thấy rằng các biến lưu trữ số nguyên là các thực thể (đối tượng) của lớp int.

Các đối tượng có thể lưu trữ dữ liệu bằng cách sử dụng các biến đối tượng tiêu chuẩn. Các trường (Fields) được gọi là *thuộc tính* (attribute) thuộc về một đối tượng hoặc lớp. Các đối tượng cũng có thể cung cấp các chức năng thông qua việc sử dụng các hàm thuộc sở hữu của lớp. Các hàm này được gọi là *phương thức* (methods) của lớp.

Thuật ngữ này rất quan trọng vì nó cho phép chúng ta phân biệt giữa các hàm và biến độc lập với các hàm và biến cụ thể của lớp hoặc đối tượng. Các trường và phương thức có thể được gọi chung là các *thuộc tính* (attributes) của lớp.

Các trường có hai loại—chúng có thể được phân loại vào bất kỳ thực thể/đối tượng nào hoặc chúng có thể được phân loại theo chính chúng (lớp). Chúng được gọi tương ứng là *biến thực thể* (instance variables) và *biến lớp* (class variables).

Lớp được phát triển bằng từ khóa class. Các trường và phương thức của lớp được hiển thị trong một loạt các thụt lề.

SELF (CHÍNH NÓ): Chỉ có một điểm khác biệt cụ thể duy nhất của các phương thức lớp so với các hàm thông thường—chúng yêu cầu một tên đầu tiên bổ sung phải được áp dụng vào đầu danh sách tham số, nhưng khi bạn gọi một phương thức, Python không gán ý nghĩa (giá trị) cho tham số này. Thuộc tính này tham chiếu đến chính đối tượng đó và theo mặc định được gọi là self.

Mặc dù rất nên sử dụng tên `self` cho tham số này—việc sử dụng bất kỳ tên nào khác chắc chắn sẽ bị phản đối. Có một số lợi ích khi sử dụng tên chung—bất kỳ người dùng nào trong phần mềm của bạn cũng có thể nhận ra nó ngay lập tức, và ngay cả các IDE tiên tiến cũng vậy.

Bạn phải tự hỏi làm thế nào Python gán ý nghĩa (giá trị) cho `self` và tại sao bạn không nên gán ý nghĩa (giá trị) cho nó. Điều này được làm rõ bằng một minh họa. Giả sử bạn có lớp `MyClass` và một đối tượng `myObject` của lớp đó. Khi bạn gọi một phương thức `myObject.method(arg1, arg2)`, điều này được Python tự động dịch thành `MyClass.method(myObject, arg1, arg2)`—đó là toàn bộ vấn đề về `self` đặc biệt này.

Điều này cũng có nghĩa là ngay cả khi bạn có một quy trình (phương thức) không có đối số, luôn có một đối số—đó là “`self`”.

CÁC LỚP (CLASSES): Lớp đơn giản nhất có thể được hiển thị trong ví dụ sau (lưu với tên `oop_simplestclass.py`).

```
class Person:
    pass # An empty block (Một khối rỗng)

p = Person ()
print (p)
```

Kết quả:

```
python oop_simplestclass.py
<__main__.Person instance at 0x10171f518>
```

Ở đây chúng ta thấy `self` đang hoạt động. Lưu ý rằng phương thức `say_hi` không nhận tham số nào nhưng vẫn có `self` trong định nghĩa hàm.

Phương thức `init` (The `init` Method):

Trong các lớp Python có một số tên phương thức có ý nghĩa đặc biệt. Ý nghĩa của quy trình (phương thức) `__init__` bây giờ đã rõ ràng. Nếu một đối tượng của một lớp được khởi tạo (ví dụ: được tạo), phương thức `__init__` sẽ được thực thi. Phương thức này hữu ích cho bất kỳ khởi tạo nào bạn muốn tạo (tức là, truyền các giá trị ban đầu cho đối tượng của bạn). Xin lưu ý dấu gạch dưới kép ở đầu và cuối của thuật ngữ.

Ví dụ (lưu với tên `oop_init.py`):

```
class Person:
    def __init__(self, name):
        self.name = name
    def say_hi(self):
        print ('Hello, my name is', self.name) # Xin chào, tên tôi là
```

```
p = Person ('Mahesh')
p.say_hi ()
# The previous 2 lines can also be written as # 2 dòng trước cũng có thể được
viết là
# Person ('Swaroop').say_hi ()
```

Kết quả:

```
python oop_init.py
Hello, my name is Mahesh
```

Ở đây, phương thức `__init__` được chỉ định với tên của một tham số (bao gồm cả ‘self’ quen thuộc). Chúng ta chỉ đang tạo một trường (field) mới có tên là ‘name’ ở đây. Lưu ý rằng có hai biến vì cả hai đều có tên là ‘name’. Cả hai đều riêng biệt. Ký hiệu dấu chấm `self.name` chỉ ra rằng thứ có tên “name” là một phần của đối tượng “self,” và ‘name’ còn lại là một biến cục bộ. Không có sự mơ hồ nào vì chúng ta nêu rõ ràng chúng ta đang đề cập đến ‘name’ nào.

Khi bạn xây dựng thực thể mới `p`, của lớp `Person`, chúng ta sử dụng tên của lớp và sau đó là các đối số trong dấu ngoặc đơn: `p = Person('Mahesh')`. Chúng ta không gọi phương thức `__init__` một cách tường minh. Đây là ý nghĩa đặc biệt của phương thức này.

Bây giờ, chúng ta có thể sử dụng trường `self.name` trong các phương thức của mình, điều này được minh chứng trong phương thức `say_hi`.

BIẾN LỚP và BIẾN ĐỐI TƯỢNG (CLASS and OBJECT VARIABLES)

Chúng ta đã đề cập đến các chức năng của các lớp và đối tượng (tức là các phương thức), và bây giờ chúng ta có thể nghe về thành phần dữ liệu. Phần dữ liệu, tức là các trường (fields), chỉ là các biến thông thường được kết nối với không gian tên (namespaces) của lớp và đối tượng. Điều này có nghĩa là những tên này chỉ tham chiếu đến các lớp và đối tượng này. Đó là lý do tại sao chúng được đặt tên là không gian tên.

Hai loại trường được phân loại tùy thuộc vào việc biến đó là thuộc tính của lớp hay đối tượng:

- **Biến lớp (Class variables)** được chia sẻ—tất cả các thực thể (instances) của lớp đó đều có thể truy cập chúng. Chỉ có 1 bản sao của biến lớp, bản sao này sẽ được sử dụng bởi tất cả các thực thể khác nếu một đối tượng thay đổi một biến lớp.
- **Biến đối tượng (Object variables)** được nắm giữ bởi mỗi đối tượng/thực thể của lớp. Trong trường hợp này, mỗi đối tượng có bản sao riêng của trường, tức là, nó không được chia sẻ trong một thực thể riêng biệt và không có cách nào kết nối với trường có cùng tên trong một thực thể khác.

Điều này sẽ dễ hiểu thông qua một ví dụ (lưu với tên `oop_objvar.py`):

```
class Robot:
    """Represents a robot, with a name.""" # Đại diện cho một robot, với một
```

cái tên.

```
# A class variable, counting the number of robots
# Một biến lớp, đếm số lượng robot
population = 0

def __init__(self, name):
    """Initializes the data.""" # Khởi tạo dữ liệu.
    self.name = name
    print (" (Initializing {})".format (self.name)) # (Đang khởi tạo ...)

    # When this person is created, the robot
    # adds to the population
    # Khi người này (robot) được tạo, robot
    # được thêm vào quần thể
    Robot.population += 1

def die (self):
    """I am dying.""" # Tôi đang chết.
    print ("{} is being destroyed!".format (self.name)) # ... đang bị phá
hủy!

    Robot.population -= 1

    if Robot.population == 0:
        print ("{} was the last one.".format (self.name)) # ... là con
robot cuối cùng.
    else:
        print ("There are still {:d} robots working.".format
(Robot.population)) # Vẫn còn ... robot đang hoạt động.

def say_hi (self):
    """Greeting by the robot.
    Yeah, they can do that.""" # Lời chào của robot. Vâng, chúng có thể
làm điều đó.
    print ("Greetings, my masters call me {}".format (self.name)) # Chào
mừng, chủ nhân của tôi gọi tôi là ...

    @classmethod
    def how_many (cls):
        """Prints the current population.""" # In ra số lượng hiện tại.
        print ("We have {:d} robots.".format (cls.population)) # Chúng ta có
... robot.

droid1 = Robot ("R2-D2")
droid1.say_hi ()
Robot.how_many ()

droid2 = Robot ("C-3PO")
droid2.say_hi ()
```

```
Robot.how_many ()
```

```
print ("\nRobots can do some work here.\n") # Robot có thể làm một số công việc ở đây.
```

```
print ("Robots have finished their work. So let's destroy them.") # Robot đã hoàn thành công việc. Vì vậy, hãy phá hủy chúng.  
droid1.die ()  
droid2.die ()
```

```
Robot.how_many ()
```

Kết quả:

```
python oop_objvar.py  
(Initializing R2-D2)  
Greetings, my masters call me R2-D2.  
We have 1 robots.  
(Initializing C-3PO)  
Greetings, my masters call me C-3PO.  
We have 2 robots.
```

```
Robots can do some work here.
```

```
Robots have finished their work. So let's destroy them.  
R2-D2 is being destroyed!  
There are still 1 robots working.  
C-3PO is being destroyed!  
C-3PO was the last one.  
We have 0 robots.
```

(Giải thích ví dụ oop_objvar.py từ Trang 29)

Đây là một ví dụ rộng (bao quát) về cách các biến lớp và biến đối tượng được minh họa. Trong trường hợp này, population (quần thể) thuộc về lớp Robot và do đó là một thuộc tính của lớp. Biến name (tên) là một biến đối tượng (biến được phân bổ bởi chính nó). Chúng ta cũng tham chiếu đến biến population là Robot.population thay vì Self.population. Chúng ta sử dụng ký hiệu self.name trong các phương thức của đối tượng để tham chiếu đến tên biến đối tượng. Hãy chú ý đến sự khác biệt cơ bản này giữa các biến của lớp và biến của đối tượng.

Cũng lưu ý rằng giá trị biến (lớp) sẽ bị che khuất bởi một đối tượng (biến đối tượng) có cùng tên!. (Lưu ý: Câu 909-910 trong tài liệu gốc có vẻ mâu thuẫn/khó hiểu) Chúng ta đáng lẽ nên sử dụng self.__class__.population thay vì Robot.population, vì bất kỳ đối tượng nào cũng được tham chiếu đến lớp của nó thông qua thuộc tính self.__class__ .

`how_many` là một hàm không thuộc về đối tượng mà thuộc về lớp. Điều này có nghĩa là, tùy thuộc vào việc chúng ta có cần biết chúng ta đang ở lớp nào hay không, chúng ta có thể mô tả nó là một phương thức động (dynamic) hoặc tĩnh (static). Chúng ta sử dụng dạng lớp (class form - `@classmethod`) vì chúng ta tham chiếu đến một thuộc tính của lớp. Chúng ta đã liệt kê nhiều cách sử dụng decorator (trình trang trí) như một công cụ của lớp. Decorators có thể được coi là các lời tắt đến một hàm được gọi bởi hàm bọc (wrapper) (ví dụ: một hàm “bọc” xung quanh một hàm để nó có thể làm điều gì đó trước hoặc sau hoạt động bên trong). Do đó, việc sử dụng decorator `@classmethod` cũng giống như gọi: `how_many = classmethod(how_many)`.

Bằng cách này, chúng ta tăng số lượng `population` lên 1 khi chúng ta thêm một robot khác. Hãy nhớ rằng các giá trị `self.name` là duy nhất cho mỗi thực thể, điều này cho thấy sự tồn tại của các biến của thực thể (biến đối tượng). Hãy nhớ rằng các biến và phương thức của cùng một đối tượng chỉ nên được tham chiếu bằng cách sử dụng `self`. Đây được coi là một thuộc tính tham chiếu.

Chúng ta cũng thấy trong phần mềm này việc sử dụng *docstrings* (chuỗi tài liệu) của lớp và quy trình (phương thức). Chúng ta có thể sử dụng `Robot.__doc__` và phương thức docstring `Robot.say_hi.__doc__`. (Lưu ý: tài liệu gốc viết `Robot.doc` và `Robot.say hi.doc`) Trong phương thức `die`, chúng ta chỉ đơn giản là giảm số đếm `Robot.population` đi 1.

Mỗi thành viên của lớp đều là *public* (công khai). Một ngoại lệ là Python sử dụng cơ chế *name-mangling* (thay đổi tên) để biến một biến thành *private* (riêng tư) một cách hiệu quả bằng cách sử dụng các tên có tiền tố gạch dưới kép như `__private_var`. Quy tắc là mọi biến chỉ để sử dụng trong lớp hoặc đối tượng sẽ bắt đầu bằng dấu gạch dưới, và tất cả các tên khác là *public* và có thể được sử dụng cho tất cả các lớp hoặc đối tượng. Quy ước (practice) này được tuân theo. Hãy nhớ rằng đây chỉ là một quy ước và không bị Python thực thi (ngoại trừ tiền tố có dấu gạch dưới kép).

KẾ THỪA (INHERITANCE)

Tái sử dụng mã là một trong những lợi ích chính của lập trình hướng đối tượng và một cách để thực hiện điều này là thông qua cơ chế kế thừa (heritage - *Lưu ý: từ đúng là “inheritance”*). Kế thừa có thể được diễn giải tốt nhất như một mối quan hệ giữa các lớp thuộc loại (type) và kiểu con (subtype).

Giả sử bạn muốn viết một chương trình theo dõi giáo viên và sinh viên trong một trường đại học. Họ có một số đặc điểm chung bao gồm tên, tuổi và địa chỉ. Họ cũng có các đặc điểm riêng, chẳng hạn như tiền lương, các khóa học và học phí của giảng viên (instructor fees) và học phí của sinh viên (student charges). Bạn có thể xây dựng và xử lý hai lớp độc lập cho mỗi loại, nhưng việc thêm một đặc điểm chung sẽ (phải) thêm vào cả hai lớp độc lập này. Điều này nhanh chóng trở nên không thể quản lý được.

Sẽ là một cách tuyệt vời để xây dựng một lớp duy nhất có tên là `SchoolMember` và sau đó có các lớp giảng viên (instructor) và sinh viên (student) thuộc lớp này, và sau đó chúng ta có thể áp dụng các tính năng khác nhau cho các kiểu con đó. Chiến lược này có

một số lợi thế. Nếu trong SchoolMember, chúng ta thêm/sửa đổi một hàm, điều này sẽ tự động được phản ánh trong các kiểu con. Ví dụ, bằng cách chỉ cần thêm nó vào lớp SchoolMember, bạn có thể tạo một trường thẻ ID (ID card area) mới cho cả giáo viên và sinh viên. Nhưng những thay đổi trong các kiểu con không ảnh hưởng đến các kiểu con khác. Một lợi ích nữa là bạn có thể tham chiếu đến một đối tượng giáo viên hoặc sinh viên như một đối tượng SchoolMember, điều này có thể hữu ích trong một số trường hợp nhất định, chẳng hạn như đếm số lượng thành viên trong trường (schoolmembers). Đây được gọi là *tính đa hình* (polymorphism), nơi một kiểu con có thể được thay thế trong bất kỳ tình huống nào mà kiểu cha được mong đợi, tức là, đối tượng có thể được coi là một thực thể của lớp cha.

Lưu ý rằng, trong trường hợp các lớp riêng biệt, chúng ta (nay đã) tái sử dụng mã của lớp và không phải sao chép nó trong các lớp khác nhau như chúng ta đã phải làm. Trong trường hợp này, lớp Schoolmember được gọi là lớp Chính (Main) hoặc lớp Cha (Superclass). Các lớp giáo viên và sinh viên được gọi là các lớp dẫn xuất (derived grades - Lưu ý: từ đúng là “*derived classes*”).

Ví dụ (lưu với tên inheritace.py):

(Lưu ý: Tên tệp trong là inheritace.py, nhưng tên tệp trong kết quả đầu ra là oop_subclass.py. Tôi sẽ sử dụng tên inheritace.py theo phần mã.)

```
class SchoolMember:
    '''Represents any school member.''' # Đại diện cho bất kỳ thành viên nào
    của trường.
    def __init__(self, name, age):
        self.name = name
        self.age = age
        print ('(Initialized SchoolMember: {})'.format (self.name)) # (Đã
khởi tạo SchoolMember: ...)

    def tell (self):
        '''Tell my details.''' # Cho biết chi tiết của tôi.
        print ('Name:"{}" Age:"{}"'.format (self.name, self.age), end=" ") #
Tên: ... Tuổi: ...

class Teacher (SchoolMember):
    '''Represents a teacher.''' # Đại diện cho một giáo viên.
    def __init__(self, name, age, salary):
        SchoolMember.__init__(self, name, age) # Gọi __init__ của lớp cha
        self.salary = salary
        print ('(Initialized Teacher: {})'.format (self.name)) # (Đã khởi tạo
Teacher: ...)

    def tell (self):
        SchoolMember.tell (self) # Gọi phương thức tell của lớp cha
        print ('Salary: "{:d}"'.format (self.salary)) # Lương: ...
```

```

class Student (SchoolMember):
    '''Represents a student.''' # Đại diện cho một học sinh.
    def __init__ (self, name, age, marks):
        SchoolMember.__init__ (self, name, age) # Gọi __init__ của lớp cha
        self.marks = marks
        print ('(Initialized Student: {})'.format (self.name)) # (Đã khởi tạo
Student: ...)

    def tell (self):
        SchoolMember.tell (self) # Gọi phương thức tell của lớp cha
        print ('Marks: "{:d}"'.format (self.marks)) # Điểm: ...

t = Teacher ('Mrs. Shrividya', 40, 30000)
s = Student ('Swaroop', 25, 75)

# prints a blank line # in một dòng trống
print ()

members = [t, s]
for member in members:
    # Works for both Teachers and Students # Hoạt động cho cả Giáo viên và
    Học sinh
    member.tell ()

```

(Lưu ý: Mã nguồn cho Student.__init__ có vẻ bị lỗi định dạng trong tài liệu gốc. Tôi đã diễn giải nó theo cách hợp lý nhất để khớp với ngữ cảnh của lớp Teacher.)

Kết quả:

```

python oop_subclass.py
(Initialized SchoolMember: Mrs. Shrividya)
(Initialized Teacher: Mrs. Shrividya)
(Initialized SchoolMember: Swaroop)
(Initialized Student: Swaroop)

```

```

Name:"Mrs. Shrividya" Age:"40" Salary: "30000"
Name:"Swaroop" Age:"25" Marks: "75"

```

TẬP TIN (FILES):

Bạn có thể mở và sử dụng tệp để đọc hoặc viết bằng cách tạo một đối tượng của lớp (class) file và sử dụng các phương thức read, readline hoặc write của nó. Việc đọc hay viết tệp phụ thuộc vào chế độ (mode) bạn đã xác định khi mở tệp. Cuối cùng, khi bạn hoàn tất tệp, bạn gọi phương thức close để thông báo cho Python rằng chúng ta đã sử dụng xong kịch bản (tệp).

Ví dụ (lưu với tên file.py):

```

poem = '''Programming is fun
When the work is done
if you wanna make your work also fun:
    use Python!'''

# Open for 'w'riting # Mở để 'w' (ghi)
f = open('poem.txt', 'w')
# Write text to file # Viết văn bản vào tệp
f.write(poem)
# Close the file # Đóng tệp
f.close()

# If no mode is specified, # Nếu không có chế độ nào được chỉ định,
# 'r'ead mode is assumed by default # chế độ 'r' (đọc) được giả định là mặc
định
f = open('poem.txt')
while True:
    line = f.readline()
    # Zero length indicates EOF # Độ dài bằng 0 cho biết EOF (Kết thúc tệp)
    if len(line) == 0:
        break
    # The 'line' already has a newline # 'line' đã có một dòng mới
    # at the end of each line # ở cuối mỗi dòng
    # since it is reading from a file. # vì nó đang đọc từ một tệp.
    print(line, end='')
# close the file # đóng tệp
f.close()

```

(Lưu ý: Tên tệp trong ví dụ đầu ra là io_using_file.py, nhưng trong phần lưu tệp là file.py. Tôi sẽ dịch theo kết quả đầu ra được cung cấp.)

Kết quả:

```

python3 io_using_file.py
Programming is fun
When the work is done
if you wanna make your work also fun:
    use Python!

```

Lưu ý rằng bằng cách đơn giản sử dụng phương thức open, chúng ta có thể tạo một đối tượng tệp mới. Thông qua việc sử dụng hàm open tích hợp và chỉ định tên tệp cùng chế độ chúng ta muốn mở tệp, chúng ta mở (hoặc tạo nó nếu nó chưa tồn tại). Chế độ có thể bao gồm chế độ đọc ('r'), ghi ('w') hoặc nối thêm ('a'). Chúng ta cũng có thể chỉ định, liệu chúng ta đang đọc, viết hoặc nối thêm ở dạng văn bản ('t') hay nhị phân ('b'). Trên thực tế, có nhiều chế độ khác có sẵn và help(open) sẽ cung cấp cho bạn thêm thông tin.

Theo mặc định, open() coi tệp là ‘tệp văn bản’ (text file) và sau đó mở nó ở chế độ ‘đọc’. Trong ví dụ của chúng ta, ban đầu chúng ta mở/tạo tệp ở chế độ văn bản và sử dụng quy

trình (phương thức) write của đối tượng tệp để ghi chuỗi biến poem của chúng ta và sau đó đóng tệp.

Đầu tiên (Tiếp theo), để xem, chúng ta mở lại cùng một tệp. Chúng ta không phải chỉ định chế độ vì chế độ mặc định là ‘đọc tệp văn bản’. Chúng ta đọc từng dòng của tệp trong một vòng lặp bằng phương thức readline. Toàn bộ dòng, bao gồm ký tự xuống dòng ở cuối dòng, được trả về. Khi chúng ta trả về một chuỗi trống, điều đó có nghĩa là chúng ta đang ở cuối tệp và thoát ra khỏi vòng lặp.

Cuối cùng, chúng ta đóng tệp. Chúng ta có thể thấy từ kết quả readline của mình rằng chương trình này đã thực sự ghi vào và đọc từ tệp poem.txt mới của chúng ta.

Tóm tắt (Summary)

Như chúng ta đã thảo luận về các kiến thức cơ bản quan trọng của python và mọi khái niệm của ngôn ngữ lập trình python đều được thảo luận với các ví dụ phù hợp. Python sẽ được sử dụng trong các phần triển khai của chương tiếp theo và để tạo các dự án thời gian thực có thể được sử dụng trong thời gian thực cho các nghiên cứu và học thuật.

Trong việc triển khai python cho các nhánh xa hơn của trí tuệ nhân tạo, có một số Thư viện và framework liên quan sẽ được thảo luận theo từng chủ đề với việc cài đặt và sử dụng chúng với các thuật toán khác nhau.